

# S4 ASSESSORIA

*Gestão e Tecnologia para PMEs*

---

Trabalho de Disciplina: Arquitetura de Software

Clean Code

TDD

BDD

Microserviços

Segurança

**Autor: Sérgio Murilo Nogueira**

Professor: Marcio Garrido | Univassoras | 2026

[s4assessoria.com.br](https://s4assessoria.com.br) | [github.com/MuriloTBS/s4-assessoria](https://github.com/MuriloTBS/s4-assessoria)

# Sumário

*Estrutura da Apresentação*

- |    |                                   |    |                          |
|----|-----------------------------------|----|--------------------------|
| 01 | Visão Geral do Projeto            | 06 | Microserviços Serverless |
| 02 | Arquitetura do Sistema            | 07 | Segurança em Camadas     |
| 03 | Clean Code                        | 08 | Resultados e Métricas    |
| 04 | TDD — Test-Driven Development     | 09 | Conclusão                |
| 05 | BDD — Behavior-Driven Development |    |                          |

# 01 • Visão Geral do Projeto

*O que é o S4 Assessoria e qual problema resolve*

## Problema

- Consultores gerenciam projetos em planilhas dispersas
- Sem visibilidade financeira do portfólio
- Prazos perdidos por falta de alertas
- Precificação feita sem metodologia

## Solução

- SPA com gestão completa de clientes e projetos
- Dashboard com gráficos de receita e status
- Alertas automáticos de prazo por e-mail (cron)
- Calculadora de precificação com fatores de complexidade

Em Produção

**s4assessoria.com.br**

Multi-tenant

**por user\_id**

Custo Infra

**R\$ 0 / mês**

# 01 • Stack Tecnológica

*Tecnologias escolhidas e justificativas*

## Frontend

React 19 + TypeScript + Vite 8

## Estilização

Tailwind CSS 4

## Roteamento

React Router DOM 7 (lazy loading)

## Gráficos

Recharts 3

## Validação

Zod 4 — schemas tipados

## Backend

Vercel Functions (Node.js serverless)

## Banco

Oracle Autonomous DB + ORDS AutoREST

## Senha

Argon2id + Pepper (server-side)

## Email

Resend API

## Monitoring

Sentry (React + Vite plugin)

## Deploy

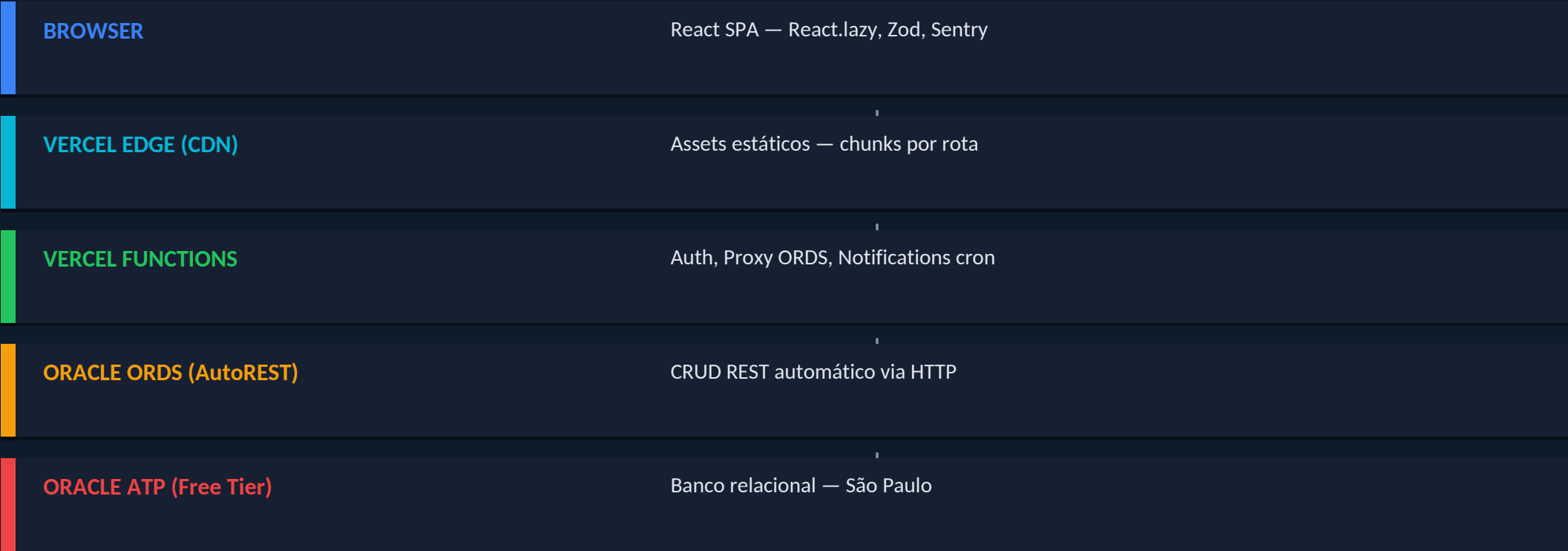
Vercel (CDN global)

## Testes UI

Vitest + Testing Library + Playwright

# 02 • Arquitetura do Sistema

Visão geral das camadas e fluxo de dados



## Princípios Arquiteturais

Separação de Responsabilidades

Stateless Serverless

Zero CORS (proxy Vercel)

HttpOnly Cookie Auth

Code Splitting (lazy)

# 03 • Clean Code

*Princípios aplicados na base de código*

## Single Responsibility

- Cada módulo tem uma única razão para mudar
- `api/auth/` cuida só de autenticação
- `api/services/` cuida só de proxy ORDS
- Hooks encapsulam toda lógica de negócio

## Nomes Expressivos

- `useAsyncAction`, `useDashboardData`, `useAdminUsers`
- `authApi`, `clientApi`, `projectApi` — domínio claro
- Sem abreviações: `'err' → 'error'`, `'u' → 'user'`
- Tipos explícitos em TypeScript

## Funções Pequenas

- Nenhum arquivo ultrapassa 200 linhas
- Cada função faz exatamente uma coisa
- Helpers isolados: `formatCurrency`, `isOverdue`
- Componentes UI genéricos e reutilizáveis

## Sem Comentários Óbvios

- Código autodocumentado por nomes claros
- Comentários só para WHY, nunca para WHAT
- Schemas Zod como documentação viva
- TypeScript como contrato de interface

# 03 • Clean Code — Na Prática

Exemplos reais do código do projeto

## Separação de camadas — api.ts

```
// Cada entidade tem sua própria API — responsabilidade única
export const clientApi = {
  async list(userId: number): Promise<Client[]> { ... },
  async create(data: Omit<Client, 'id' | 'created_at'>): Promise<Client>
{ ... },
  async delete(id: number): Promise<boolean> { ... },
}

export const projectApi = {
  async list(userId: number): Promise<Project[]> { ... },
  async get(id: number): Promise<ProjectWithSteps> { ... },
}
```

## Hook com responsabilidade única — useAsyncAction.ts

```
// Encapsula loading + error para qualquer ação assíncrona
export function useAsyncAction() {
  const [working, setWorking] = useState(false)
  const [error, setError] = useState<string | null>(null)

  async function execute<T>(fn: () => Promise<T>): Promise<T | undefined> {
    setWorking(true); setError(null)
    try { return await fn() }
    catch (e) { setError(e instanceof Error ? e.message : 'Erro') }
    finally { setWorking(false) }
  }

  return { working, error, execute }
}
```

## Métricas Clean Code

Maior arquivo

< 200 linhas

Componentes UI

5 reutilizáveis

Hooks customizados

4 especializados

Camadas isoladas

UI / Hook / API

Tipagem

TypeScript estrito

Schemas

Zod + inferência

# 04 • TDD — Test-Driven Development

Ciclo Red → Green → Refactor aplicado ao projeto

## RED

Escrever o teste

Teste falha — comportamento ainda não existe

## GREEN

Implementar

Código mínimo para o teste passar

## REFACTOR

Refatorar

Melhorar sem quebrar os testes existentes

### Suíte de Testes — Resultados

- ✓ schemas.test.ts > loginSchema > aceita credenciais válidas
- ✓ schemas.test.ts > loginSchema > rejeita email inválido
- ✓ schemas.test.ts > registerSchema > rejeita nome muito curto
- ✓ schemas.test.ts > parametersSchema > rejeita margem acima de 100
- ✓ utils.test.ts > formatCurrency > formata valores em BRL
- ✓ utils.test.ts > formatDate > retorna traço para undefined
- ✓ utils.test.ts > isOverdue > retorna true para data no passado
- ✓ useAsyncAction.test.ts > captura erro em error state
- ✓ useAsyncAction.test.ts > retorna resultado da função executada
- ✓ Button.test.tsx > variant danger aplica classe text-red-400

Test Files 4 passed (4) | Tests 28 passed (28) | Duration ~700ms

4

Test Files

28

Total Tests

100%

Coverage

~700ms

Duration



# 05 • BDD — Behavior-Driven Development

Cenários em linguagem natural executáveis com Playwright + Cucumber

## Feature: Autenticação

```
Feature: Autenticação
  Como usuário do sistema
  Quero fazer login com minhas credenciais
  Para acessar o painel de gestão

  Scenario: Login com credenciais inválidas
    Given estou na página de login
    When preencho o email "invalido@teste.com" e senha
    "senhaerrada"
    And clico em Entrar
    Then vejo a mensagem "Email ou senha incorretos"

  Scenario: Validação de senha curta
    Given estou na página de login
    When preencho o email "teste@teste.com" e senha "123"
    And clico em Entrar
    Then vejo a mensagem "pelo menos 6 caracteres"
```

## Feature: Painel Admin

```
Feature: Painel Admin
  Como administrador do sistema
  Quero controlar o acesso de usuários
  Para garantir segurança da plataforma

  Scenario: Cadastro gera conta pendente
    Given estou na página de login
    When clico em "Criar conta"
    And preencho nome, email e senha válidos
    And clico em "Criar conta"
    Then vejo "Aguarde aprovação do administrador"

  Scenario: Admin vê painel de administração
    Given estou autenticado como administrador
    When acesso /admin
    Then vejo o título "Painel Admin"
```

## Por que BDD?

### Linguagem ubíqua

Cenários legíveis por PO, dev e QA

### Documentação viva

Testes são a especificação

### E2E automático

Playwright executa os cenários no browser

### Regressão segura

Mudanças detectadas imediatamente

# 06 • Microserviços Serverless

*Funções independentes com responsabilidade única*

## `api/auth/login.js`

- Argon2+Pepper verify
- Migração SHA-256 → Argon2
- Rate limiting (Vercel KV)
- Set HttpOnly Cookie

## `api/auth/register.js`

- Hash Argon2 da senha
- Criação de organização
- Status inicial PENDING

## `api/auth/me.js`

- Valida cookie HMAC
- Verifica expiração (24h)
- Retorna dados do usuário

## `api/auth/logout.js`

- Limpa cookie HttpOnly
- Max-Age=0 no servidor

## `api/auth/forgot-password.js`

- Token HMAC-SHA256
- Expiração 1 hora
- Envio via Resend API

## `api/auth/reset-password.js`

- Valida token + timestamp
- Novo hash Argon2
- Nunca revela se email existe

## `api/notifications/deadline.js`

- Cron diário 09h BRT
- Filtra prazos 3 dias
- Email personalizado Resend

## `api/services/*.js`

- Proxy autenticado ORDS
- Valida x-s4-internal-key
- Repassa para Oracle ATP

# 06 • Microserviços — Isolamento de Falhas

*Cada serviço falha de forma independente*

*Princípio fundamental: nenhuma função depende das outras para existir.*

## Resend (email) indisponível

- Login continua funcionando normalmente
- Recuperação de senha falha com HTTP 500
- Notificações de prazo não são enviadas
- Todos os outros endpoints operam normalmente

## Vercel KV (rate limiting) indisponível

- Fallback automático para Map em memória
- Rate limiting ainda funciona por instância
- Login nunca retorna erro por causa do KV
- Degradação graciosa sem impacto ao usuário

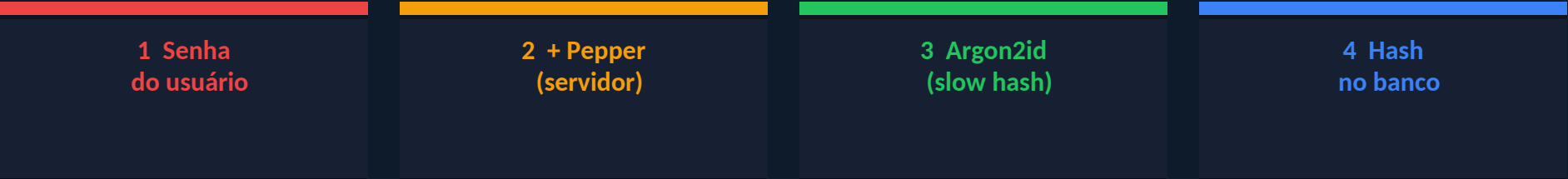
## Oracle ORDS retorna timeout

- Vercel Function retorna HTTP 500 ao cliente
- UI exibe 'Tentar novamente' com botão de retry
- Sentry captura o erro e alerta o time
- Sessão do usuário não é afetada

# 07 • Segurança em Camadas

Argon2id + Pepper — Padrão ouro em hashing de senha

## Fluxo de armazenamento de senha



```
// api/auth/login.js - verificação e migração automática de legado
if (isArgon2Hash(storedHash)) {
  valid = await argon2Verify(storedHash, password + PEPPER)
} else {
  // Legacy SHA-256 - verifica e migra na primeira entrada
  valid = storedHash === sha256Legacy(password)
  if (valid) {
    const newHash = await argon2Hash(password + PEPPER)
    await updateUserHash(user.id, newHash) // migração transparente
  }
}
```

### Por que Argon2id?

- ✓ Vencedor do Password Hashing Competition
- ✓ Resistente a ataques de GPU e ASIC
- ✓ Ajuste de custo computacional
- ✓ Uso de memória configurável
- ✓ Padrão recomendado pelo OWASP
- ✓ Pepper = inutiliza o banco sem a chave

## Demais Controles de Segurança

|                                                               |                                                          |                                                            |
|---------------------------------------------------------------|----------------------------------------------------------|------------------------------------------------------------|
| <b>HttpOnly Cookie</b><br>Token HMAC não acessível por JS     | <b>Rate Limiting</b><br>10 tentativas / IP / 15 min (KV) | <b>HMAC + Expiração</b><br>Token de reset expira em 1 hora |
| <b>Anti-enumeração</b><br>Resposta sempre HTTP 200 no esqueci | <b>Headers HTTP</b><br>HSTS, X-Frame, CSP, nosniff       | <b>Internal API Key</b><br>Chave entre frontend e backend  |

# 07 • Segurança — HttpOnly Cookie e Headers HTTP

Proteção contra XSS, CSRF e ataques de rede

## Sessão baseada em HttpOnly Cookie (substituição do sessionStorage)

```
// api/auth/login.js - cookie criado no servidor, invisível ao JavaScript
const token = createSessionToken(user.id, user.email, user.name)
res.setHeader('Set-Cookie', [
  `s4_session=${token}`,
  'HttpOnly',          // JavaScript não consegue ler - imune a XSS
  'Secure',            // Apenas HTTPS
  'SameSite=Strict',   // Imune a CSRF cross-site
  'Max-Age=86400',      // 24 horas
  'Path=/',
].join('; '))

// api/auth/me.js - valida em todo page refresh sem expor o token ao JS
function verifySessionToken(token) {
  const [payload, sig] = token.split('.')
  const expected = createHmac('sha256',
    SESSION_SECRET).update(payload).digest('base64url')
  if (sig !== expected) return null // assinatura inválida
  if (Date.now() - data.iat > 86400000) return null // expirado
  return JSON.parse(Buffer.from(payload, 'base64url').toString())
}
```

### Defesa em profundidade

HTTPS / HSTS

HttpOnly Cookie

Rate Limiting

HMAC assinado

Argon2+Pepper

### Headers HTTP (vercel.json)

**Strict-Transport-Security**

HSTS 2 anos + preload

**X-Content-Type-Options**

Bloqueia MIME sniffing

**X-Frame-Options**

Previne clickjacking

**X-XSS-Protection**

Proteção XSS browsers

**Referrer-Policy**

Controla headers Referer

**Permissions-Policy**

Bloqueia câmera/micro

# 08 • Resultados e Métricas

Evidências quantitativas da qualidade do projeto



# 09 • Conclusão

*O que o projeto demonstra sobre Arquitetura de Software*

## Clean Code

Código legível, módulos com responsabilidade única, sem comentários desnecessários. Qualquer desenvolvedor consegue entender e evoluir o sistema.

## TDD

28 testes unitários documentam o comportamento esperado. Mudanças são validadas automaticamente — regressões são detectadas antes do deploy.

## BDD

Cenários Gherkin em linguagem natural descrevem os fluxos de negócio. Stakeholders lêem e validam sem precisar entender código.

## Microserviços

8 funções serverless independentes com responsabilidade única. Falha isolada, deploy independente, escala automática por demanda.

## Segurança

Defesa em 5 camadas: HTTPS, HttpOnly Cookie, Rate Limiting, HMAC assinado, Argon2+Pepper. Cada camada endereça um vetor de ataque específico.

*"Arquitetura de software não é sobre ferramentas — é sobre decisões que permitem o sistema crescer sem se tornar um problema."*

S4 Assessoria | Arquitetura de Software | Univassoras

# Obrigado!

---

*S4 Assessoria — Gestão e Tecnologia para PMEs*

**Autor:** Sérgio Murilo Nogueira

**Professor:** Marcio Garrido

**Instituição:** Univassoras

**Disciplina:** Arquitetura de Software

**Projeto:** [s4assessoria.com.br](http://s4assessoria.com.br)

**GitHub:** [github.com/MuriloTBS/s4-assessoria](https://github.com/MuriloTBS/s4-assessoria)

Clean Code

TDD

BDD

Microserviços

Segurança